TaxTag: A Zero-Knowledge Tax Filing Protocol for Decentralized Finance

Yihsuan Wu[1,a)] Kazue Sako[1,b)]

Abstract: Taxpayers declare their crypto-asset tax themselves. Centralized service providers must identify their customers, who are the taxpayers, and report their transactions to tax authorities. The authorities verify the declarations against these reports. In decentralized finance (DeFi) this model runs into a trade-off. If the reporting duty keeps reaching only centralized service providers, then no report exists for DeFi and the authority cannot verify the correctness or completeness of what the taxpayer declares. If the duty instead extends to DeFi protocols, then their providers must identify taxpayers, and pseudonymity is destroyed for the protocol and the authority alike. We propose TaxTag, a zero-knowledge tax filing protocol that verifies what taxpayers declare while linking them to nothing on the chain.

Keywords: Decentralized Finance, Tax Evasion, Zero-knowledge Proof, Unlinkability

1. Introduction

Centralized exchanges (CEX) in Japan take on a reporting duty from January 2026 [1]. They identify each customer and report their transactions, so the tax authority can verify a taxpayer's declaration against the report. However, decentralized finance (DeFi) has no such intermediary. A DeFi transaction runs through a smart contract on a public blockchain, where taxpayers transact under pseudonymous addresses, and no intermediary stands in its path with a duty to comply or to report [2]. Therefore, enforcement in DeFi runs into a trade-off. For example, if the reporting duty keeps reaching only CEXs, then a taxpayer can declare a lower price than executed, or omits a transaction entirely, and the tax authority cannot verify the declaration since it has no report to check against. Alternatively, if the authority extends the reporting duty to DeFi protocols, then their providers must identify taxpayers, and every taxpayer's pseudonymous addresses become linked to their real identity. Verification and pseudonymity pull against each other, and a solution must deliver both.

No existing system resolves the trade-off. zkLedger [3] audits closed bank ledgers and Provisions [4] proves CEX solvency, both for a designated verifier over committed data, and neither covers activity on a public chain. zkTax [5] proves statements about a return the taxpayer chooses to disclose, so nothing shows the numbers are real or complete. Privacy Pools [6] binds its statement to real on-chain funds but cannot show that nothing was withheld.

We propose TaxTag, a zero-knowledge tax filing protocol that resolves the trade-off. It replaces third-party reporting with the taxpayer's own declaration, which carries a proof of correctness and completeness. The tax authority issues each taxpayer one signed *tax tag* and publishes the list of chains TaxTag covers. TaxTag runs as one smart contract per listed chain, and a taxpayer may trade on any number of listed chains. The taxpayer uses the same tag on every listed chain, and keeps one *tax ledger* per chain, a local append-only list of the taxable transactions recorded there.

For each taxable transaction, the taxpayer proves to TaxTag's smart contract that they hold a valid tag, without revealing the tag itself. TaxTag's smart contract then executes the trade and writes the amounts and prices on-chain, while the taxpayer appends the transaction to the local ledger. A hidden identifier derived from the tax tag binds the on-chain entry to the ledger entry, and only the taxpayer can compute it. The figures therefore come from execution, not from the taxpayer, and nothing on the chain shows whose transaction it is. Once per year the taxpayer makes one filing that covers every chain. A zero-knowledge proof shows that the declared proceeds, costs, and income equal the year's tax rules evaluated over the tag's complete ledgers and the CEX's annual report. These are the three numbers a Japanese return reports for miscellaneous income. The tax authority learns only these three numbers and a hiding commitment to the balances carried into the next year.

We make three contributions.

C1. We define three security properties, two for verification and one for pseudonymity, and six functional requirements from a worked two-year scenario in the Japanese jurisdiction (Section 2).

C2. We propose TaxTag, a zero-knowledge tax filing protocol that meets the functional requirements (Section 3).

[1] Waseda University
[a)] yihsuanwu84@suou.waseda.jp
[b)] kazuesako@waseda.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

We run the scenario through the protocol and argue the properties through attack traces (Section 4).

C3. We evaluate TaxTag's recording and filing costs and demonstrate practicality at deployment scale (Section 5).

2. Security Properties and Protocol Requirements

This section sets a two-year scenario, then states the three security properties and the six functional requirements. Formal definitions and proofs are future work.

2.1 Running Scenario

Scope setting. The scenario is based on the Japanese jurisdiction. The National Tax Agency (NTA) names three kinds of taxable transactions for a crypto-asset, selling it for fiat, exchanging it for another crypto-asset, and paying for goods or services [7]. However, paying for goods or services stays outside the scope, since it does not happen in DeFi. Reward receipt, lending, collateral, and liquidation also stay outside, because Japanese law has not settled their treatment [8]. The tax is calculated annually, and the return is filed by a fixed date.

Tax calculation method. Unless the taxpayer elects another calculation method, every sale of a year uses one annual average unit cost per asset, the total-average method [9]. Annual averages need only per-asset yearly totals and never a transaction order, which keeps one filing cheap across many chains.

Scenario. Alice uses one CEX and two DeFi protocols, on chains X and Y, over two tax years, trading three crypto-assets ETH, G, and N. Table 1 fixes the history and shows the calculation results, with fees omitted. Steps t_i are year-1 and u_i year-2 transactions. A swap sells one asset and buys another at the trade's JPY market value. Japanese law taxes it like a sale at that value, and the bought asset is booked at that value, shown as "paid" in the table.

We use Alice's G to show the calculation. In year 1 she gets 1,000 G and pays 600 yen for them. One G therefore costs $600/1{,}000 = 0.6$ yen. This unit cost, marked † in the table, is fixed only at year-end filing, since the average needs the whole year's totals. Every sale of G in year 1 uses this one price, whatever day it happened. Her sale of 400 G has proceeds 650, the JPY value received. Its cost is the units sold times the unit cost, $400 \times 0.6 = 240$. Its gain is the proceeds minus the cost, $650 - 240 = +410$. The filing sums the year's gains into the income and carries unsold units into the next year, her 600 G at cost $600 \times 0.6 = 360$. In general, every sale of a year y uses the unit cost $\mathsf{avg}_y = (V_y^{\text{open}} + C_y^{\text{acq}})/(Q_y^{\text{open}} + Q_y^{\text{acq}})$. All four symbols are per asset. Q_y^{open} and V_y^{open} are the units and the JPY value carried into year y. Q_y^{acq} and C_y^{acq} are the units bought and the JPY paid during year y.

We use Alice's N to show how the carry works. Year 2 opens with the carried 900 N at cost 450, and u_1 adds 1,100 N for 650. N's year-2 unit cost is $(450 + 650)/2{,}000 = 0.55$, merging both years.

During a year, each transaction only adds to its asset's yearly totals, the units and yen bought and the units and yen sold so far. Moving units between Alice's CEX account and her on-chain addresses is not a taxable transaction and creates no record, since each asset has one set of totals wherever its units sit.

2.2 Security Properties

Unforgeability and completeness deliver verification, ruling out false declarations and omitted transactions. Unlinkability keeps pseudonymity, ruling out identity links.

Unforgeability. No price or amount entering the filing may differ from the taxable transaction as executed. This covers the records TaxTag's smart contract writes, the CEX report, and the balances carried from last year.

Completeness. Completeness has two halves. First, every taxable transaction in DeFi is *recorded* by TaxTag. Second, every transaction in the tag's tax ledgers is *opened* at filing, so no accepted filing can leave one out.

Unlinkability. Using TaxTag, at recording or at filing, lets no party, including the tax authority, link the taxpayer's identity to an on-chain address. A link between the taxpayer's identity and one address through the CEX's records, for example a deposit to or a withdrawal from the CEX, lies outside the property, since TaxTag does not create it.

2.3 Functional Requirements

Each requirement names the scenario step that exposes it. Section 4 checks all six on the worked example.

F1: Sealed prices and amounts (t_2). The executed amounts and the transaction-time JPY value of every taxable transaction are written on-chain by TaxTag's smart contract at execution.

F2: Cross-chain filing (t_3). One filing covers all deployed chains.

F3: Cross-channel filing (t_1, t_4). One filing merges the CEX-reported and the DeFi-recorded amounts.

F4: Policy-complete filing (filing_1). The filing applies the tax rules the tax authority published for that year. It outputs the declared proceeds, costs, and income.

F5: Cross-year continuity (u_1, u_2, filing_2). The closing balances an accepted filing emits, per asset in quantity and value, are the next filing's opening balances. A record after the year's boundary belongs to the next year.

F6: Multi-asset filing (ETH, G, N). One filing covers every asset the year touches.

3. The TaxTag Protocol

This section introduces the participants, the on-chain components, and the trust assumptions, walks the yearly workflow, defines the data structures and TaxTag's steps, and states the two circuits and their checks.

3.1 Participants and Trust Model

TaxTag involves four participants and two on-chain components.

- **Taxpayer.** Holds one tag issued by the tax authority.

© 2026 Information Processing Society of Japan

Table 1 Running scenario and its total-average accounting. Plain numbers are JPY, and a dash means not applicable.

Time	Where	Transaction	Sold	Unit cost†	Cost	Bought	Proceeds	Gain
Year 1								
t_1	CEX	buy	–	–	–	2 ETH (paid 1,000)	–	–
t_2	Chain X	swap	1 ETH	500	500	1,000 G (paid 600)	600	+100
t_3	Chain Y	swap	1 ETH	500	500	900 N (paid 450)	450	−50
t_4	CEX	sell	400 G	0.6	240	–	650	+410
filing_1	income = 460; carry to year 2: 600 G (cost 360), 900 N (cost 450); no ETH left							
Year 2								
u_1	Chain X	swap	600 G	0.6	360	1,100 N (paid 650)	650	+290
u_2	CEX	sell	800 N	0.55	440	–	600	+160
filing_2	income = 450; carry to year 3: 1,200 N (cost 660); no G left							

Keeps one append-only tax ledger per deployed chain, locally. Records every taxable transaction and files once per year.

- **Tax authority (TA).** Verifies the taxpayer's identity and issues the tag. Maintains the tag revocation list. Publishes the filing circuit that encodes the year's tax rules. Verifies filings off-chain and countersigns the closing state of an accepted filing, the balances carried into the next year.
- **Exchange (CEX).** A centralized exchange that reports each customer's annual aggregates to the TA and gives the customer a copy.
- **DeFi protocols.** The protocols where TaxTag recording is deployed. Each confirms what the taxpayer asked for and hands the trade to the TaxTag contract, which executes and records it.
- **TaxTag contract.** TaxTag's smart contract, one per deployed chain. It verifies recording proofs, executes the transfer, and maintains the on-chain structures of Section 3.3.
- **Price oracle.** Supplies transaction-time JPY values to the TaxTag contract at execution.

The trust model has five assumptions.

- **A1.** The TA acts honestly at issuance. It verifies the taxpayer's identity, signs the tag, and records who holds what tag together with the registration anchor, a hash of the taxpayer's secret. Its credential signing key and the key of its recognized TaxTag contract stay uncompromised.
- **A2.** The blockchains, the DeFi protocols, and the TaxTag contracts execute correctly, the zkSNARK trusted setups are correct, and the TaxTag contract holds the TA's recognized public key as deployed.
- **A3.** DeFi protocols reject any trade that does not prove an unrevoked tag.
- **A4.** The price oracle supplies transaction-time JPY values correctly.
- **A5.** The CEX's annual report is correct.

3.2 Workflow

Figure 1 shows one full tax year. The first phase, issuance, happens only once in a tag's life. The later phases repeat every year. At issuance the taxpayer proves their identity and hands over a registration anchor, and the TA issues the signed tag and writes the anchor on each deployed chain. Trading then runs all year. CEX trades need no TaxTag recording, since the CEX knows its customer. A DeFi protocol passes each swap to the TaxTag contract, which executes and records it in one atomic call. The year ends, and each chain's TaxTag contract pins its year-end state. The CEX sends its annual report to the TA and to the taxpayer. After the year ends, the taxpayer checks every record of the year against the pinned state and combines the checks into one small proof, the folding of Section 3.4. They file this proof, π_{file}, with the declared items and the closing-state commitment cs_y by the filing deadline, while new-year trading continues. The TA verifies with its published circuit and countersigns cs_y, which opens the next year.

3.3 Data Structures and Steps

Notation. λ is the security parameter and $x \xleftarrow{\$} X$ uniform sampling. H is a collision- and preimage-resistant hash, modeled as a random oracle only where hiding or unlinkability is argued. The commitment $H(m \,\|\, \rho)$ with $\rho \xleftarrow{\$} \{0,1\}^\lambda$ is binding and hiding. A sparse Merkle tree (SMT) proves membership and non-membership, since an empty-leaf path under a published root proves absence [10]. A zk-SNARK provides knowledge soundness and zero knowledge [11].

Tag, secret, and anchor. The TA samples the tag $tag \xleftarrow{\$} \{0,1\}^\lambda$ and signs it, giving the credential σ_{TA}. During issuance the taxpayer samples a secret $S \xleftarrow{\$} \{0,1\}^\lambda$ and hands the anchor $H(S)$ to the TA. The TA writes the anchor into each chain's anchor table (AT), an SMT at index $H(tag)$, under its recognized key. The TaxTag contract refuses a second write at an occupied index, so not even the TA can replace an anchor later. The hidden identifier is $ID_{\text{tax}} = H(tag, S)$. Only the taxpayer knows S, so no other party can compute or recognize ID_{tax}. Ownership is layered. Knowledge of S shows the registered holder, and σ_{TA} is a second factor, so a leaked S alone lets nobody record under the tag. Registration involves no taxpayer transaction, so no taxpayer address is exposed.

Records. A record R_i captures one taxable transaction in two parts. The *intent part* is what the taxpayer asked for, the tuple of the operation type, the asset sold, the asset bought, and the amount offered. The *executed part* E_i is what execution produced, the amounts actually moved in and out, the fee, and the oracle JPY price. The TaxTag contract writes E_i, because these values exist only at execution. A record has two legs, the asset sold and the asset bought. The intent's asset identifiers assign each leg to its asset's totals, and the report's rows carry the same identifiers. The tax

© 2026 Information Processing Society of Japan

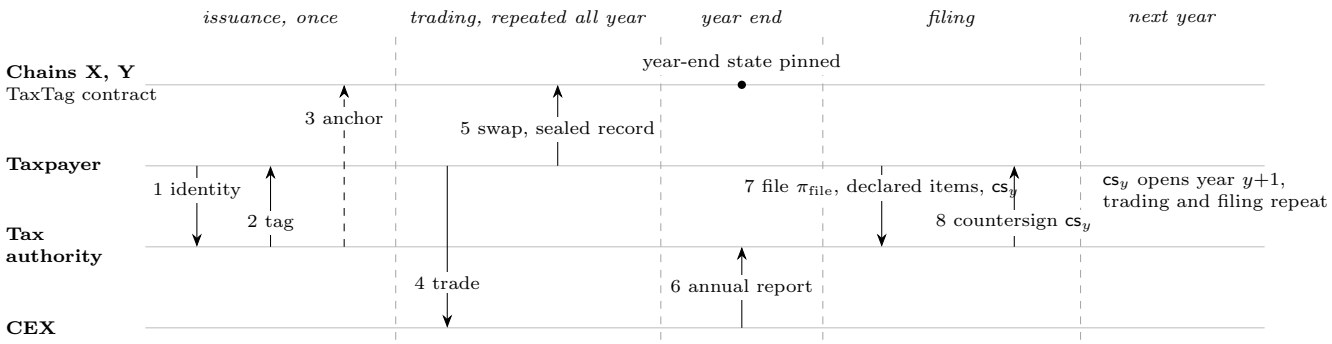


Fig. 1 One tax year. Arrows touching the chains row are public transactions. No message carries a taxpayer address to the TA or an identity to a DeFi protocol. TaxTag uses only the CEX report's per-asset totals.

ledger for a chain is the sequence of these records, and i is the position of R_i in it, counted over the tag's lifetime.

What the chain sees. Only the taxpayer knows the tax ledger's contents. TaxTag derives four hashes per record. The intent digest is $TX_i = H(\text{intent part})$. The fingerprint $\mathsf{fp}_i = H(\mathsf{fp}_{i-1} \,\|\, TX_i)$, with $\mathsf{fp}_0 = H(ID_{\text{tax}})$, chains the whole history, so nothing can later be reordered or dropped. The record commitment $rec_i = H(ID_{\text{tax}} \,\|\, \mathsf{fp}_i \,\|\, i \,\|\, \rho_i)$ hides the state under a fresh blinding ρ_i. The nullifier $NF_i = H(ID_{\text{tax}} \,\|\, ACaddr \,\|\, i)$ is deterministic in the position i, where $ACaddr$ is the TaxTag contract's address. The chain sees only rec_i and NF_i. The fingerprint fp_i stays inside the witness, hidden by rec_i, since a published fingerprint would let anyone link a taxpayer's records. At acceptance the TaxTag contract forms the leaf $\ell_i = H(rec_i \,\|\, cid \,\|\, block \,\|\, idx \,\|\, E_i)$ from its own execution context. cid names the chain, and consensus assigns $block$ and idx, which the taxpayer cannot choose.

On-chain structures. The TaxTag contract maintains three SMT-based lists and one accumulator. The record commitment list (RCL) is the append-only accumulator of the leaves ℓ_i. The used-nullifier list (UNL) holds every consumed NF_i. The anchor table (AT) holds the registration anchors. The tag revocation list (TRL) holds a leaf 1 at $H(tag)$ once the TA revokes the tag under its recognized key. The boundary block B is the chain's last block of the tax year, and the TaxTag contract pins its snapshot roots there. It checkpoints its roots at the first write of each new year. The pinSnapshot call finalizes the checkpointed values, so the pinned roots are exactly the boundary roots even if new-year records arrive first. The pinned roots also assign records to years, since a record appended after B has no path under $\mathsf{rt}_{\text{RCL}}(B)$ and its position forces it into the next year's segment.

Recording step. The taxpayer submits the trade request with the proof π_{rec} and the outputs rec_i and NF_i. The DeFi protocol confirms the intent by placing it into the proof's public inputs, so the proof verifies only for the trade it actually executes, and passes the call to the TaxTag contract. The contract verifies the proof, executes the transfer, writes E_i into the leaf, appends ℓ_i to the RCL, and consumes NF_i into the UNL, all in one atomic call. Any failure reverts everything, so no execution happens without its record. The taxpayer stores R_i locally.

Filing step. The taxpayer proves the filing statement over the year's segment of each chain's ledger. Three inputs beyond the ledgers enter the statement. The pinned snapshot roots anchor the year. The CEX aggregates are the per-asset totals of the annual report, public inputs the TA checks against its own copy under A5. Only these totals enter the statement. The report's remaining fields, transfers and year-end balances, stay outside as ordinary audit material. Addresses the CEX knows stay outside TaxTag. The countersigned closing state of the previous filing supplies the opening balances. The TA verifies off-chain with its published verifier and learns the declared proceeds, costs, and income, and the commitment cs_y. Nothing is consumed or appended, so filings are repeatable for amendments and audits. An amended year re-issues its closing state, so later years re-file in order. Filing cost is one year's records, never the lifetime.

Continuity step. An accepted filing ends its year with cs_y, a hiding commitment to the closing balances and, per chain, the last opened position and fingerprint. The balances form one balance tree with a leaf per asset. The TA countersigns cs_y. The next filing opens the countersigned state as its opening balances and covers only the new year's records. This is Table 1's carryover line, in TaxTag form. The commitment stays in the TA's acceptance record, and its opening stays only with the taxpayer. The blinding derives from the secret S, so the taxpayer can recompute a lost opening, and the TA's record restores a lost commitment value. **Revocation.** Every recording proof shows the tag's TRL leaf is empty, so a revoked tag records nothing further. Its tax ledgers stay openable, since revocation must not erase the duty to file. The revoked tag files one last time and emits its final closing state. The successor tag's first filing opens that countersigned state as its opening balances. The TA's registry ties both tags to the same taxpayer. Year handoff and tag handoff are the same mechanism.

3.4 Circuit Statements and Checks

TaxTag has exactly two proof statements. The recording statement C_{rec} is verified on-chain at every taxable transaction, and the filing statement off-chain once per year. Public inputs form each statement, and private inputs form the

© 2026 Information Processing Society of Japan

witness.

Recording statement. Public are the confirmed intent, rec_i, NF_i, the current roots, the TA's recognized key, $ACaddr$, and the submitting address. Private are tag, S, σ_{TA}, the counter and salts, the previous record's opening, and the Merkle paths.

Six checks form the relation over the four hashes of Section 3.3. The previous-commitment check opens rec_{i-1} under $\mathsf{rt}_{\mathrm{RCL}}$. The intent digest check binds TX_i to the confirmed intent. The extension forms fp_i and rec_i from them. The nullifier check forms NF_i and proves it a non-member of the UNL. The non-revocation check shows $ID_{\mathrm{tax}} = H(tag, S)$ with the TRL leaf at $H(tag)$ empty. The credential check verifies σ_{TA} on tag under the recognized key. Together they show one thing. The submitted record extends this tag's single ledger by exactly one position, under a tag the TA issued and has not revoked, with an intent the DeFi protocol confirms.

The first record of a chain's ledger swaps the previous-commitment check for the anchor check, the AT opening of $H(S)$ at index $H(tag)$, and starts from $\mathsf{fp}_0 = H(ID_{\mathrm{tax}})$. Re-initialization is impossible, since a second first record reuses NF_1. Every proof binds the TaxTag contract and the submitting address, so proofs replay nowhere else. On acceptance the contract forms ℓ_i from its own execution context, appends it, and consumes NF_i, within the recording step's single atomic call. The circuit cost is constant per record, fixed hashes and paths and one signature check.

Filing statement. Figure 2 lists the five checks of C_{file}. The circuit is the NTA's total-average worksheet, compiled, since this paper handles one jurisdiction and one method. The TA verifies with the key of the circuit it published for the year, so filing under a stale or altered circuit is impossible. A law change means the TA compiles and publishes a new circuit, and every tax ledger already recorded stays provable.

Why the end-of-ledger check. Nullifiers in payment systems stop the reuse of an old state [12]. A filing prover cheats the other way, opening a strict prefix and hiding the recent, unfavorable tail. Check 3 turns the same nullifier set into an exhaustiveness proof. If any record beyond n existed by the boundary, NF_{n+1} would already be consumed. The same check settles an inactive chain, which contributes only the non-membership of its next nullifier, so every filing covers every deployed chain in the same shape. NF_{n+1} stays private. A public value would let the TA watch the UNL until the next recording consumes it and link the filer to that transaction's address.

Realization by folding. A year can reach thousands of records, and a monolithic filing circuit fixes a hard record bound at setup. We therefore prove filing by incrementally verifiable computation [13]. A small *step circuit* checks one record and updates a running state. Folding runs this step once per record, but it makes no proof per step. It keeps one running claim that all steps so far were correct. One final small proof then compresses the whole run. The cost is seconds per step, minutes for a full year, and one constant-size proof, measured in Section 5. Folding runs after the boundary, because a membership path under $\mathsf{rt}_{\mathrm{RCL}}(B)$ exists only once B has pinned the root. A step folded earlier could only open against a root the prover chooses, and no public input would ever check that root. The folded state therefore carries the pinned root, each step opens its leaf under it, and the final circuit checks the state root against the public $\mathsf{rt}_{\mathrm{RCL}}(B)$. So every step provably used the root the TA reads from the chain. Total average keeps the step lean, since only the yearly totals of Section 2.1 enter.

The running state holds the per-asset totals in a small Merkle tree, the totals tree. Think of it as 256 numbered boxes, one box per asset. The asset identifier is the box number, and the identifier's bits are the turns on the path from the box to the root. Each record updates the two boxes its legs touch, the sold asset's box and the bought asset's box. At Alice's t_2, ETH's box gains a disposal of 1 unit for 600, and G's box gains an acquisition of 1,000 units for 600. One asset has exactly one box, no per-asset circuitry exists, and one step circuit serves any asset set. *Rep* and the carried balances enter at the final circuit, published in a few slot capacities, where the totals merge. The step carries check 2, and the final circuit adds checks 1, 3, 4, and 5. Zero knowledge comes from the final compressing proof over a blinded folded instance. The prototype uses the stock compressor without this blinding, as Section 5 notes. The recording statement, whose verification is gas-critical, stays a small pairing-based circuit.

Filing statement C_{file} (year y; verified off-chain by the TA).
Public inputs: the tag tag; per chain c: the snapshot roots at B and $ACaddr^{(c)}$; the CEX report totals Rep, per asset the units bought and sold with their JPY amounts, checked by the TA against its copy of the report; the countersigned cs_{y-1}; the declared items; the new commitment cs_y.
Private inputs: S; the opening of cs_{y-1}, per chain the last position k and fingerprint fp_k, and the carried balance root; per chain the openings $\{(R_i, \rho_i, block_i, idx_i, path_i)\}_{i=k+1}^{n}$ and NF_{n+1} with its non-membership path; the used assets' leaves, totals under the folded totals root and balances under the carried balance root, with their paths; the blinding of cs_y.
The circuit proves:
(1) **(Opening state)** cs_{y-1} opens to the per-chain (k, fp_k) and the carried balance root. A chain's first filing instead verifies the anchor under $\mathsf{rt}_{\mathrm{AT}}(B)$ and starts from $k = 0$ and $\mathsf{fp}_0 = H(ID_{\mathrm{tax}})$;
(2) **(Openings)** for $i = k{+}1..n$: TX_i digests R_i's intent part, $\mathsf{fp}_i = H(\mathsf{fp}_{i-1} \,\|\, TX_i)$, $rec_i = H(ID_{\mathrm{tax}} \,\|\, \mathsf{fp}_i \,\|\, i \,\|\, \rho_i)$, $\ell_i = H(rec_i \,\|\, cid_c \,\|\, block_i \,\|\, idx_i \,\|\, E_i)$, and ℓ_i is in the RCL under $\mathsf{rt}_{\mathrm{RCL}}(B)$;
(3) **(End of ledger)** $NF_{n+1} = H(ID_{\mathrm{tax}} \,\|\, ACaddr^{(c)} \,\|\, n{+}1)$ is a non-member of the UNL under $\mathsf{rt}_{\mathrm{UNL}}(B)$, kept private;
(4) **(Worksheet)** for each of at most M used assets, open its totals leaf under the folded totals root and its balance leaf under the carried root. Merge opening balance, *Rep* purchases, and sums. Unit cost is JPY value over units and applies to every sale, in *Rep* and in the sums. Rebuilding both trees from opened leaves and empty defaults must return both roots, so no asset hides. The results sum to the declared items;
(5) **(Closing state)** cs_y commits the per-chain (n, fp_n) and the root of the closing balance tree.

Fig. 2 The filing statement. The TA learns the declared items and the hiding commitment cs_y, which it countersigns on acceptance.

© 2026 Information Processing Society of Japan

4. Worked Example and Attack Analysis

This section runs the scenario of Table 1 through TaxTag, naming each functional requirement at the step that meets it. Each attack trace then names the check that stops it.

4.1 The Scenario Through the Protocol

Issuance. Alice proves her identity to the TA once and receives her tag. The TA writes her anchor into the anchor table of chains X and Y. No address of hers is exposed.

Recording year 1. At t_2 Alice submits the swap on chain X with a recording proof. The DeFi protocol confirms her intent, 1 ETH in. The TaxTag contract executes, writes the executed part, 1,000 G out, the fee, and the value 600, and appends the leaf as position 1 of her chain-X ledger, consuming its nullifier. Alice declared no value anywhere. The 600 that later enters her gain is contract-written, which is F1. At t_3 the same happens on chain Y, position 1 of an independent ledger. The two chains never coordinate, which is the setting of F2. The CEX trades t_1 and t_4 create no TaxTag record and go into the annual report.

Filing year 1. The year ends and each chain pins its snapshot roots. The report *Rep* states that Alice bought 2 ETH for 1,000 and sold 400 G for 650. Alice folds one step per record against the pinned roots. The chain-X step re-opens t_2 and updates ETH's and G's boxes, the update of Section 3.4. The chain-Y step does the same for t_3, touching ETH and N. The final circuit starts from the empty opening state through the anchor check, opens the three used assets' totals, and merges *Rep* into them, computing exactly the filing_1 row of Table 1. ETH's unit cost is 500, G's is 0.6, and the gains are $+100$, -50, and $+410$. The proceeds sum the three sales, $600 + 450 + 650$. The costs sum the same sales, $500 + 500 + 240$. The declared items are proceeds 1,700, costs 1,240, and income 460. Three assets, one proof, which is F6. ETH's cost comes only from *Rep* and its proceeds only from the ledgers, and each cost enters the totals once, which is F3. The circuit is the worksheet the TA published for the year, which is F4. The proof also shows the end of each ledger, the next nullifier unconsumed at the boundary, and commits the closing state, 600 G at cost 360 and 900 N at cost 450, with each ledger's last position. The TA checks *Rep* against its copy, verifies with its published key, and countersigns.

Filing year 2. At u_1 Alice's chain-X ledger grows to position 2. The CEX sale u_2 goes into the next report. Filing_2 opens the countersigned closing state, opens only chain X's record 2, and shows chain Y's ledger unchanged. Its next nullifier is still unconsumed, a verifiable nothing-to-report. The carried G cost 360 becomes u_1's cost. N's totals merge the carried 450 with the new 650 into 0.55. The result is proceeds 1,250, costs 800, and income 450, and the new closing state, 1,200 N at cost 660. The chain of accepted states is F5.

4.2 Attack Traces

Each trace modifies one element of the honest run, grouped under the property it targets.

Against unforgeability.

- **ATK1, forged price.** Alice claims t_2 was worth 500, not 600, to shrink the gain. She stated no value at recording, the TaxTag contract wrote 600 into the leaf. A filing that opens 500 recomputes a leaf that is not under the pinned root, so the openings check fails.
- **ATK2, fake tree.** Alice folds her steps against a private tree holding a forged leaf with value 550. The folded state carries her root, and the final circuit compares it with the pinned root the TA reads from the chain. They differ, so the proof never verifies.
- **ATK3, double-claimed cost.** Alice wants the 400 G sold at the CEX to cost their full 600 while the same 600 stays in the on-chain totals. One circuit computes both channels from one set of totals per asset. There is no second place to claim a cost, so the attack has no statement to forge.
- **ATK4, inflated carryover.** In year 2 Alice opens the carried G cost as 600 instead of 360. The closing state is a binding commitment the TA countersigned. Opening it to other values needs a hash collision, so the opening-state check fails.

Against completeness.

- **ATK5, hidden record.** Alice files year 1 without t_2, claiming her chain-X ledger is empty. The end-of-ledger check then needs the ledger's first nullifier unconsumed at the boundary. It was consumed at recording, so the filing is rejected. Any hidden tail fails the same way, since the nullifier of the next position is deterministic. A restarted or parallel fresh ledger also fails, since its first record reforms that consumed nullifier, and the write-once anchor table admits no second identifier under her tag.
- **ATK6, unrecorded execution.** Alice asks a DeFi protocol to swap without recording. The recording step's single atomic call forbids it, so no execution there exists without its record under A3.
- **ATK7, year shifting.** Alice tries to move a December record into year 2, or a January record into year 1. The first fails like ATK5, its nullifier consumed before the boundary. The second has no path under year 1's pinned root and lands in filing_2.

Unlinkability. No trace breaks unlinkability, and the argument is structural. Recording exposes the public trade data and values that are hashes over the hidden identifier. The hashes are random to every observer. The DeFi protocol sees a trade and no identity. The filing is verified off-chain. The next nullifier stays private, for the reason Section 3.4 gives. Two taxpayers with equal declared items look the same to the TA. Links Alice creates at the CEX are outside the property, as Section 2.2 scopes.

Trust boundary. Unforgeability and completeness hold under the TA's issuance honesty, A1, and the report's correctness, A5, which no cryptography enforces. Unlinkability needs no such trust. Records carry hashes over the hidden

© 2026 Information Processing Society of Japan

Table 2 TaxTag contract interface and gas, IR-optimized solc. Ranges come from tree path lengths. Dashes are view functions. The taxpayer never calls the contract, and no address enters any TaxTag structure.

Caller	Function	Effect	Gas
TA	writeAnchor	AT entry at $H(tag)$, write-once	120k–212k
TA	revokeTag	TRL entry at $H(tag)$	97k
DeFi	record, first	verify, add ℓ_i, use NF_i	2.49–2.55M
DeFi	record, update	same, every later record	1.91–2.07M
		of which verifyProof	293k
anyone	currentRoots	read	–
anyone	anchorAt	read	–
anyone	pinSnapshot	pin the year's roots at B	94k

identifier and never the tag, so the TA learns no address, only who holds the tag.

5. Evaluation

We implemented the recording statement, the TaxTag contract, and the folding filing prover. We measured two kinds of cost, the taxpayer's proving and the gas of every TaxTag contract call. All numbers come from one consumer laptop with an 8-core Ryzen AI 7 H 350 and 32 GB of RAM, in ordinary use, warm after a first prove that pays the proving-key load, and every measurement is reproducible by script.[*1]

Implementation. The recording statement compiles to its two circuits in circom [14], as rank-one constraint systems (R1CS), proven with Groth16 over BN254 using circomlib's Poseidon, sparse Merkle tree, and EdDSA components.[*2] The TaxTag contract is Solidity, exercised on an in-process EVM with on-chain Poseidon. The recording flow runs end to end, real proofs against live contract state, every tree root checked against an independent implementation. The filing prover folds the step circuit with Nova [13] and CycleFold [15] and compresses with a final Groth16 proof. Table 2 lists the contract interface.

Recording costs. Table 3 gives every proof TaxTag asks of the taxpayer, its circuit size, and its proving cost. Each SMT level costs about 253 constraints per authenticated path, so depth is the main dial. At depth 64 the taxpayer proves a recording in four to five seconds and under 1.8 gigabytes of memory with the slowest available prover, snarkjs in JavaScript, and the proof is 256 bytes of calldata. The gap between the 293k verification and the full record call in Table 2 is on-chain Poseidon hashing for the RCL append and the UNL insert, so hashing dominates, not verification. At mainnet prices this points deployment at a rollup, where the same call costs cents.

Filing costs. The step circuit re-verifies one record and updates its two totals leaves in 18,708 constraints. Folding one record takes 0.8 to 0.9 seconds from 100 to 1,000 records, the spread being thermal drift. The folded instance verifies in under 80 milliseconds at either scale. The final compressing proof is one fixed circuit of 791,960 constraints whatever the ledger holds. It takes 8 seconds and a 2.7 gigabyte peak to produce and 6 milliseconds for the TA to verify, constant in the ledger length. Folding 100 alternating swaps returns exactly the hand-computed totals tree. This final circuit is the stock compressing verifier. The worksheet finalization, the report merge, and the closing-state commitment run in the harness, and their circuit cost is estimated, not measured.

Table 3 Taxpayer-side circuits and proving cost. Constraints are non-linear R1CS at SMT depth 64. The step opens a depth-32 RCL and the depth-8 totals tree. At depth 64 it grows by about 8,100 constraints. Recording uses snarkjs, the slowest prover. Memory is the peak working set.

Proof	When	R1CS	Time	Memory
recording, first	once per ledger	63,228	4.3–5.1 s	1.8 GB
recording, update	later records	53,941	4.1 s	1.8 GB
filing step, folded	per record, after B	18,708	0.8–0.9 s	0.6 GB
final proof	once per filing	791,960	8 s	2.7 GB

A full-year filing demonstration. We generated an independent, reproducible year of 1,000 transactions over 100 assets. Of these, 819 are DeFi records on two chains, 444 on X and 375 on Y, and 181 are CEX trades that enter as the report totals *Rep*. Each chain's ledger folds under the same step circuit into one 384-byte proof. The folded totals root matches an independent computation on both chains. The worksheet merges every asset's totals with *Rep* and returns the declared items. Proceeds are 400,397,727 yen, costs 400,241,959, and income 155,768. The closing state commits the 100 balances and both cursors. The TA checks two 384-byte proofs and learns three yen amounts and one commitment. It learns no transaction detail, not the asset, not the DeFi protocol, and not the date. This exercises F2's per-chain inputs, F3, and F6 at scale, with the merge in the harness.

6. Conclusion

TaxTag resolves the trade-off, preventing tax evasion in DeFi under Japan's default total-average method without breaking unlinkability. We stated three security properties, two for verification and one for pseudonymity, and six functional requirements from a worked two-year scenario. TaxTag meets them with sealed on-chain records, per-asset yearly totals, and one zero-knowledge filing proof per year. Running the scenario through the protocol checked all six requirements, and attack traces argued the three properties. A circom and Nova prototype records in 54k to 63k constraints, folds a record in under a second, and verifies a year in 6 milliseconds. Future work covers different tax calculation methods, other jurisdictions' tax rules, a wider set of taxable transactions such as reward receipt and liquidation, multi-tag holdings, recovery after a lost secret, and formal proofs of the security properties.

[*1] The benchmark artifact, circuits and scripts, is at `https://github.com/JqnWu27/taxtag-css26-bench`.

[*2] circom, snarkjs, and circomlib at `https://github.com/iden3`, the folding library sonobe at `https://github.com/privacy-ethereum/sonobe`.

References

[1] 国税庁. 暗号資産等報告枠組み（CARF）, 2024. Effective 1 January 2026. `https://www.nta.go.jp/taxes/shiraberu/kokusai/carf/index.htm`.

[2] Omri Marian. Are cryptocurrencies super tax havens? *Michigan Law Review First Impressions*, 112:38–48, 2013.

[3] Neha Narula, Willy Vasquez, and Madars Virza. zkLedger: Privacy-preserving auditing for distributed ledgers. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*, pages 65–80. USENIX Association, 2018.

[4] Gaby G. Dagher, Benedikt Bünz, Joseph Bonneau, Jeremy Clark, and Dan Boneh. Provisions: Privacy-preserving proofs of solvency for Bitcoin exchanges. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*, pages 720–731. ACM, 2015.

© 2026 Information Processing Society of Japan

[5] Alex Berke, Tobin South, Robert Mahari, Kent Larson, and Alex Pentland. zkTax: A pragmatic way to support zero-knowledge tax disclosures. arXiv:2311.13008, 2023.

[6] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Schär, and Ameen Soleimani. Blockchain privacy and regulatory compliance: Towards a practical equilibrium. *Blockchain: Research and Applications*, 5(1):100176, 2024.

[7] 国税庁. 暗号資産等に関する税務上の取扱いについて（FAQ）, 2024. https://www.nta.go.jp/information/release/kokuzeicho/2018/faq/index.htm.

[8] 上村和紀. DeFi（分散型金融）の税務上の取扱いについての一考察. 税務大学校論叢, (113):1–74, 2024. https://www.nta.go.jp/about/organization/ntc/kenkyu/ronsou/113/01/index.htm.

[9] 国税庁. 暗号資産等に関する税務上の取扱い及び計算書について（令和7年12月）, 2025. https://www.nta.go.jp/publication/pamph/shotoku/kakuteishinkokukankei/kasoutuka/.

[10] Rasmus Dahlberg, Tobias Pulls, and Roel Peeters. Efficient sparse Merkle trees: Caching strategies and secure (non-)membership proofs. In *Secure IT Systems – NordSec 2016*, pages 199–215. Springer, 2016.

[11] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326. Springer, 2016.

[12] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy (SP)*, pages 459–474. IEEE, 2014.

[13] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Advances in Cryptology – CRYPTO 2022*, pages 359–388. Springer, 2022.

[14] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. Circom: A circuit description language for building zero-knowledge applications. *IEEE Transactions on Dependable and Secure Computing*, 20(6):4733–4751, 2023.

[15] Abhiram Kothapalli and Srinath Setty. CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. Cryptology ePrint Archive 2023/1192, 2023.

© 2026 Information Processing Society of Japan